

SECTION B No. 2 RNN LSTM PROBLEM

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
df=pd.read_csv('../DATASET/No.2/COVID-19.csv')
```

a. Implement preprocessing steps on the given dataset

selecting the data

```
df = df[['India New cases']]
```

plotting the selected data to show the trends

```
plt.figure(figsize=(10, 5))
plt.plot(df['India New cases'])
plt.title('India New COVID-19 Cases Trend')
plt.xlabel('Month Index')
plt.ylabel('New Cases')
plt.show()
```

normalizing the data between 0~1

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(df)
```

split the data into 80% training 20% testing

```
from sklearn.model_selection import train_test_split

train, test = train_test_split(scaled_data, test_size=0.2, random_state=0)
```

define `create_dataset()` function

```
import numpy as np

def create_dataset(dataset, time=1):
    data_X, data_Y = [], []
    for i in range(len(dataset)-time-1):
        a = dataset[i:(i+time), 0]
        data_X.append(a)
        data_Y.append(dataset[i + time, 0])
    return np.array(data_X), np.array(data_Y)
```

create `train_X`, `train_y` and `test_X`, `test_y` using `create_dataset()`

```
time_step = 1

train_X, train_Y = create_dataset(train, time_step)
test_X, test_Y = create_dataset(test, time_step)
```

reshape the `train_X`, `train_y` and `test_X`, `test_y` so it can be an input for LSTM

```
train_X = np.reshape(train_X, (train_X.shape[0], train_X.shape[1], 1))
test_X = np.reshape(test_X, (test_X.shape[0], test_X.shape[1], 1))
```

b. By using Tensorflow Keras API version 2, build a LSTM model with 3 LSTM layers, and each layer with 25 units/nodes. After each LSTM layer, add a dropout layer with 20% dropout rate. As the last layer, apply a dense layer with a correct number of units.

```
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import LSTM
from keras.layers import Dropout

model = Sequential()

model.add(LSTM(25, return_sequences=True, input_shape=(time_step, 1)))
model.add(Dropout(0.2))
model.add(LSTM(25, return_sequences=True))
model.add(Dropout(0.2))
model.add(LSTM(25))
model.add(Dropout(0.2))
model.add(Dense(1))

model.summary()
```

c. Train and evaluate the performance of your model.

```
# Compiling the LSTM
model.compile(optimizer='adam', loss='mean_squared_error')

# Fitting the LSTM with the training set
model.fit(train_X, train_Y, epochs=100, batch_size=1, verbose=1)
```

inverse transform the data

```
train_predict = model.predict(train_X)
test_predict = model.predict(test_X)

train_predict = scaler.inverse_transform(train_predict)
train_Y = scaler.inverse_transform([train_Y])
test_predict = scaler.inverse_transform(test_predict)
test_Y = scaler.inverse_transform([test_Y])
```

evaluate the model's performance

```
from sklearn.metrics import mean_squared_error
import math

train_score = math.sqrt(mean_squared_error(train_Y, train_predict[:, 0]))
test_score = math.sqrt(mean_squared_error(test_Y, test_predict[:, 0]))

print('Train RMSE:', train_score)
print('Test RMSE:', test_score)
```

plot the predicted data vs true data

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))
plt.plot(test_Y[0], label='True Data')
plt.plot(test_predict[:, 0], label='Predicted Data')
plt.title('COVID-19 New Cases: True vs Predicted')
plt.xlabel('Time Step')
plt.ylabel('New Cases')
plt.legend()
plt.show()
```

